Guía de laboratorio — TP6: Desarrollar un pequeño servidor

Redes de Computadoras · UNC – FCEFyN · Los Tíos Networks · Dos PCs propias: Linux (servidor y captura) y Windows (cliente)

PC	Rol	Qué corre
LINUX Ubuntu 26.04	**Servidor** y **captura**	`python3 -m http.server 8000` y Wireshark
WINDOWS	**Cliente**	Navegador, `curl` y `certutil` (desde **cmd**)

IP de la Linux (`IP_LINUX`): _____ IP de la Windows (`IP_WINDOWS`): _____

En todos los comandos, reemplazar `IP_LINUX` e `IP_WINDOWS` por estas direcciones.

¿WiFi o cable?

Las dos opciones sirven. HTTP, TCP e IP funcionan igual sobre WiFi que sobre Ethernet, y esta parte de la consigna solo pide dos computadoras que se alcancen por IP.

	A) WiFi — las dos en la misma red	B) Cable directo Linux ↔ Windows
Configuración	Ninguna: el router asigna las IPs por DHCP	IP fija en las dos: `192.168.50.1` (Linux) y `192.168.50.2` (Windows)
Cuándo conviene	En casa, con las dos PCs conectadas a la misma red	Si la red WiFi aísla a los clientes (redes de facultad o de invitados) o si quieren repetir el esquema cableado de la Parte 2
Interfaz a capturar	`wlp2s0`	`enp3s0`
A tener en cuenta	Hay tráfico de fondo de otros dispositivos; se resuelve con filtros (Paso 3)	La Windows necesita puerto RJ45 o un adaptador USB. Sirve el cable derecho de la Parte 1: las placas actuales ajustan solas los pares de transmisión y recepción (Auto MDI-X)

Recomendado: A (WiFi) si lo hacen en casa. Si el ping del Paso 1 no funciona, pasar a B.

> **PARA EL INFORME:** la parte UDP se hizo con cable y switch junto a otro grupo. En esta actividad hay que aclarar que se usaron dos PCs propias, qué conexión se eligió y por qué.

0 Preparar → 1 Red → 2 Servidor → 3 Wireshark → 4 Navegador → 5 curl → 6 Extra (opcional) → 7 Guardar → 8 Restaurar

IMPRESCINDIBLE DURANTE LA PRÁCTICA: guardar la captura `.pcapng` y sacar las capturas de pantalla del navegador, de `curl` y `certutil` en la Windows y de la terminal del servidor en la Linux. Eso no se puede reconstruir después. El análisis del Paso 7 se puede terminar más tarde con el archivo guardado.

Paso 0 — Preparar las dos PCs

antes, con internet

LINUX 1. Instalar Wireshark

```
sudo apt update
sudo apt install wireshark
```

Cuando pregunte *"¿Deberían los usuarios sin privilegios de superusuario poder capturar paquetes?"*, responder **Sí**. Si se pasó de largo: `sudo dpkg-reconfigure wireshark-common`.

LINUX 2. Darse permiso para capturar

```
sudo usermod -aG wireshark $USER
```

Cerrar sesión y volver a entrar (o reiniciar). Comprobar con `groups`: tiene que aparecer `wireshark`.

> **POR QUÉ:** así Wireshark captura sin abrirlo con `sudo`. Ejecutar toda la interfaz gráfica como administrador no es recomendable.

LINUX 3. Crear la carpeta con tres tipos de archivo

```
TP6="$HOME/Escritorio/facu/redes_de_computadoras/Trabajos Prácticos/Los-Tios-
Networks/Trabajos Practicos - Practico/TP6"
mkdir ~/servidor_tp6 && cd ~/servidor_tp6
echo "Prueba del servidor HTTP - Los Tios Networks" > archivo.txt
cp "$TP6/RC - TP N6.pdf" archivo.pdf
cp "$TP6/Multimedia/01_configuracion_ipv4.png" imagen.png
ls -l
```

> **POR QUÉ:** con tres tipos distintos se ve que el servidor manda cada archivo con un `Content-type` diferente (`text/plain`, `application/pdf`, `image/png`). **No poner un `index.html`**: si lo hay, el servidor muestra esa página en lugar del listado. Se usa una carpeta aparte porque el servidor publica **todo** lo que hay en ella, subcarpetas incluidas, para cualquiera en la red. Nunca levantarlo en la carpeta personal.

LINUX 4. Revisar el firewall

```
sudo ufw status            # esperado: "Estado: inactivo" (hoy lo está)
sudo ufw allow 8000/tcp    # solo si dijera "activo"
```

WINDOWS 5. Preparar la terminal y la carpeta de descarga

Abrir **Símbolo del sistema** (Win+R → `cmd`) y ejecutar:

```
curl --version
mkdir C:\tp6
```

> **ATENCIÓN:** usar **cmd**, no PowerShell. En Windows PowerShell, `curl` es un alias de `Invoke-WebRequest` y la opción `-O` da error. Si se usa PowerShell, escribir `curl.exe`.

Paso 1 — Conectar y configurar la red

Opción A — WiFi

1. LINUX Conectarse a la red y ver la IP: `ip -br -4 addr show wlp2s0` → por ejemplo `wlp2s0 UP 192.168.1.7/24`. La IP la asigna el router y puede cambiar: anotar la de ese día como `IP_LINUX`.
2. WINDOWS Conectarse a la **misma** red y ejecutar `ipconfig` → sección "Adaptador de LAN inalámbrica Wi-Fi" → "Dirección IPv4". Anotarla como `IP_WINDOWS`.
3. Comprobar que las dos IPs compartan los tres primeros números (misma red `/24`).

Opción B — Cable directo

1. Conectar el cable entre las dos PCs.
2. LINUX Crear un perfil de red aparte con IP fija (no modifica la configuración del WiFi):

```
sudo nmcli con add type ethernet ifname enp3s0 con-name tp6-cable \
     ipv4.method manual ipv4.addresses 192.168.50.1/24
sudo nmcli con up tp6-cable
ip -br -4 addr show enp3s0       # → enp3s0 UP 192.168.50.1/24
```

3. WINDOWS Win+R → `ncpa.cpl` → clic derecho en **Ethernet** → Propiedades → **Protocolo de Internet versión 4 (TCP/IPv4)** → Propiedades → "Usar la siguiente dirección IP":
   - Dirección IP: `192.168.50.2`
   - Máscara de subred: `255.255.255.0`
   - Puerta de enlace y DNS: **vacíos**

   Aceptar y comprobar con `ipconfig` (sección "Adaptador de Ethernet").

> **POR QUÉ:** la subred `192.168.50.0/24` es distinta de la del WiFi de casa (`192.168.1.0/24`), así ninguna PC duda por qué interfaz enviar los paquetes, y el WiFi puede seguir conectado para tener internet. No hace falta gateway porque las dos PCs están en la misma subred. Si la Windows muestra una IP `169.254.x.x` en Ethernet, la IP fija no se aplicó: es la dirección que Windows se asigna sola cuando no hay DHCP.

Prueba de conectividad (ambas opciones)

WINDOWS `ping IP_LINUX` → tienen que llegar respuestas.

- El ping en sentido contrario (Linux → Windows) puede fallar porque el firewall de Windows bloquea por defecto el ping entrante. **No afecta esta actividad**: la Windows es cliente y todas sus conexiones son salientes.
- Si por WiFi no hay respuesta, la red puede estar aislando a los clientes. Probar con el hotspot del celular, que suele permitir la comunicación entre dispositivos conectados, o pasar a la opción B.

Paso 2 — Levantar el servidor

inciso 1

LINUX

```
cd ~/servidor_tp6
python3 -m http.server 8000
```

Salida esperada: `Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...`. El `0.0.0.0` significa que escucha en **todas** las interfaces, tanto WiFi como cable.

- [ ] En **otra pestaña** de la terminal (la primera queda ocupada con el servidor): `ss -ltn | grep 8000` → `LISTEN … 0.0.0.0:8000`.
- [ ] Probarlo en la propia Linux: `http://localhost:8000`.

> **POR QUÉ:** el 8000 es un puerto mayor a 1024, así que no requiere `sudo` y no choca con el 80. La terminal muestra un **log** de cada pedido (IP del cliente, ruta y código de respuesta), que sirve como evidencia.

Paso 3 — Arrancar Wireshark ANTES de acceder

LINUX

1. Abrir Wireshark (sin `sudo`) y elegir la interfaz: `wlp2s0` (opción A) o `enp3s0` (opción B). **No usar `any` ni `lo`**: `any` muestra "Linux cooked capture" en lugar de Ethernet II, y `lo` solo ve el tráfico interno de la propia PC.
2. Opcional, recomendado con WiFi: en el campo de filtro de captura de la pantalla inicial, escribir `host IP_WINDOWS`. Así solo se guarda el tráfico entre las dos PCs, el archivo queda chico y sin tráfico de otros dispositivos.
3. Iniciar la captura y aplicar el filtro de visualización `tcp.port == 8000`.

> **POR QUÉ:** se arranca antes para no perder el inicio de la conexión (*three-way handshake*). Se captura en el servidor porque es un extremo de la conexión: ve todos los paquetes que entran y salen. Aunque se use WiFi, Wireshark muestra las tramas como **Ethernet II**: en modo normal la placa WiFi le entrega las tramas al sistema ya convertidas a formato Ethernet. Para ver los encabezados 802.11 haría falta el modo monitor, que no se necesita acá.

Paso 4 — Acceso desde el navegador

inciso 2

1. WINDOWS Entrar a `http://IP_LINUX:8000/`, escribiendo el `http://` explícito. Debe aparecer **"Directory listing for /"** con los tres archivos.
2. WINDOWS Descargar uno. Si se hace clic en el PDF, el navegador lo abre en lugar de descargarlo: usar clic derecho → **"Guardar enlace como…"**.
3. LINUX Mirar la terminal del servidor. Se deberían ver líneas como estas:

```
IP_WINDOWS - - [..] "GET / HTTP/1.1" 200 -
IP_WINDOWS - - [..] "GET /favicon.ico HTTP/1.1" 404 -
IP_WINDOWS - - [..] "GET /archivo.pdf HTTP/1.1" 200 -
```

- El **404 de `/favicon.ico`** es normal: el navegador pide por su cuenta el ícono de la pestaña y la carpeta no lo tiene.
- Si se recarga la página puede aparecer un **304 Not Modified** (caché del navegador). Para evitarlo: **Ctrl+F5** o una ventana de incógnito.

Paso 5 — Descarga con curl

inciso 3

WINDOWS En cmd:

```
cd C:\tp6
curl -v -O http://IP_LINUX:8000/archivo.pdf
certutil -hashfile archivo.pdf SHA256
```

Con `-v` se ven los encabezados enviados (`>`) y recibidos (`<`). En la prueba previa con esta misma Linux salió:

```
> GET /archivo.pdf HTTP/1.1
> User-Agent: curl/8.x
> Accept: */*
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.14.4
< Content-type: application/pdf
< Content-Length: 369766
```

LINUX Calcular el hash del original. Tiene que coincidir con el que dio `certutil`:

```
sha256sum ~/servidor_tp6/archivo.pdf
```

> **POR QUÉ:** `-v` deja evidencia del pedido y la respuesta HTTP. Que los hashes coincidan demuestra que TCP entregó el archivo completo y sin errores, a diferencia de UDP, que no garantiza ni la entrega ni el orden.

Paso 6 — Experimento extra con el servidor apagado

opcional, 2 minutos

1. LINUX Cortar el servidor con **Ctrl+C**, dejando Wireshark capturando.
2. WINDOWS Ejecutar `curl -v http://IP_LINUX:8000/` → debería fallar con *"Couldn't connect to server"*.
3. LINUX En Wireshark se espera `SYN` → `RST, ACK`: el sistema de la Linux avisa que nadie escucha en ese puerto. Es posible que el par se repita un par de veces, porque Windows reintenta la conexión antes de darse por vencido.
4. LINUX Volver a levantar el servidor si hace falta repetir algo.

> **POR QUÉ:** es el equivalente en TCP del *ICMP Port Unreachable* que se observó con UDP (pregunta 5c). Si ufw estuviera activo, en lugar del RST se verían SYN sin respuesta. Hoy ufw está inactivo, así que se espera el RST.

Paso 7 — Guardar y analizar

inciso 4 · se puede terminar después

LINUX Guardar **fuera** de `~/servidor_tp6`, para no publicarlo con el servidor:

- [ ] **Archivo → Guardar como** → `~/Escritorio/tp6_servidor_http.pcapng` (captura completa).
- [ ] **Archivo → Exportar paquetes especificados** → "Solo mostrados" (filtro `tcp.port == 8000`) → `~/Escritorio/tp6_servidor_http_filtrado.pcapng`.

Qué buscar	Cómo verlo	Qué demuestra
`SYN` → `SYN, ACK` → `ACK` al puerto 8000	Filtro `tcp.flags.syn == 1`	TCP establece la conexión antes de enviar datos (UDP no)
Opciones del SYN: MSS 1460, escalado de ventana, SACK	Expandir TCP → Options	Negociación de parámetros (1460 = 1500 de MTU − 20 de IP − 20 de TCP)
`GET … HTTP/1.1` → `HTTP/1.0 200 OK`	Filtro `http`	El cliente pide en HTTP/1.1, pero Python responde en HTTP/1.0
Segmentos de datos y sus ACK	Lista de paquetes	Confirmación de recepción y control de flujo
`FIN` enviado primero por el servidor	Filtro `tcp.flags.fin == 1`	Con HTTP/1.0 el servidor cierra la conexión después de cada respuesta
Pedido y respuesta en texto plano	Clic derecho → Seguir → Flujo TCP	HTTP no va cifrado
Diagrama de la conexión	Estadísticas → Gráfico de flujo	Apertura, datos y cierre en una sola imagen
Capas del tráfico	Estadísticas → Jerarquía de protocolos	Ethernet → IPv4 → TCP → HTTP
Archivos transferidos (opcional)	Archivo → Exportar objetos → HTTP	Cualquiera que capture puede reconstruir los archivos

- Si la columna *Protocol* muestra solo `TCP` y nunca `HTTP`: clic derecho en un paquete → **Decodificar como…** → puerto TCP 8000 → HTTP. Wireshark reconoce HTTP por el número de puerto y puede no tener el 8000 entre los predeterminados.
- Con el navegador es normal ver **varias conexiones TCP**, incluso alguna sin datos: los navegadores abren conexiones por adelantado y en paralelo. Con `curl` hay una sola, así que esa captura es la más clara para explicar.

Paso 8 — Cerrar y restaurar

- [ ] **LINUX** Cortar el servidor con **Ctrl+C**.
- [ ] **LINUX** Opción B: borrar el perfil del cable con `sudo nmcli con delete tp6-cable`.
- [ ] **WINDOWS** Opción B: volver IPv4 de Ethernet a **"Obtener una dirección IP automáticamente"** y DNS automático.
- [ ] **LINUX** Si se abrió el puerto en ufw: `sudo ufw delete allow 8000/tcp`.
- [ ] Opción A: no hay nada que restaurar, porque las IPs las asignó el router.

Si algo falla

Síntoma	Causa probable	Qué hacer
El ping desde la Windows no responde (WiFi)	Las PCs están en redes distintas o la red aísla a los clientes	Comparar las IPs; probar con el hotspot del celular o pasar a la opción B
El ping desde la Windows no responde (cable)	IP mal cargada, perfil no activo o cable	`ip -br -4 addr show enp3s0`, `sudo nmcli con up tp6-cable`, `ipconfig`; revisar las luces del puerto
La Windows muestra `169.254.x.x` en Ethernet	La IP fija no se guardó	Repetir la configuración IPv4 en `ncpa.cpl`
El ping Linux → Windows falla	El firewall de Windows bloquea el ping entrante	Es normal y no afecta la actividad
El navegador da "tiempo de espera agotado"	ufw activo en la Linux, o IP equivocada	`sudo ufw status`; si está activo, `sudo ufw allow 8000/tcp`
"Conexión rechazada"	El servidor no está corriendo o el puerto es otro	Revisar la terminal del servidor y `ss -ltn | grep 8000`
"Address already in use"	El puerto 8000 ya está ocupado	Usar otro puerto (ej.: `8001`) y anotarlo para el informe
Wireshark no muestra interfaces o da error de permisos	El usuario no está en el grupo `wireshark` o no se volvió a iniciar sesión	`groups`; repetir `sudo usermod -aG wireshark $USER` y cerrar sesión
Se ve "Linux cooked capture" en lugar de Ethernet II	Se capturó en la interfaz `any`	Capturar en `wlp2s0` o `enp3s0`
La columna Protocol muestra solo TCP	Wireshark no asocia el puerto 8000 con HTTP	Decodificar como… → puerto TCP 8000 → HTTP
Aparece 304 en lugar de 200	Caché del navegador	Ctrl+F5 o ventana de incógnito
`curl -O` da error	Se está usando PowerShell	Usar cmd o escribir `curl.exe`

Capturas de pantalla para el informe

siguen la numeración de Multimedia/

- ☐ **LINUX** `07_red_linux.png` — salida de `ip -br -4 addr` (y de `nmcli` si se usó cable)
- ☐ **WINDOWS** `08_red_windows.png` — salida de `ipconfig`
- ☐ **WINDOWS** `09_ping_windows_a_linux.png` — ping exitoso
- ☐ **LINUX** `10_directorio_y_servidor.png` — `ls -l` de la carpeta y el servidor corriendo
- ☐ **WINDOWS** `11_navegador_listado.png` — "Directory listing for /"
- ☐ **WINDOWS** `12_navegador_descarga.png` — el archivo descargado desde el navegador
- ☐ **LINUX** `13_log_servidor.png` — log del servidor con los 200 y el 404
- ☐ **WINDOWS** `14_curl_descarga.png` — salida de `curl -v` y de `certutil`
- ☐ **LINUX** `15_hash_original.png` — salida de `sha256sum`
- ☐ **LINUX** `16_wireshark_handshake.png` — SYN, SYN-ACK, ACK con las opciones del SYN desplegadas
- ☐ **LINUX** `17_wireshark_http_get_200.png` — el GET y la respuesta `HTTP/1.0 200 OK`
- ☐ **LINUX** `18_wireshark_flujo_tcp.png` — Seguir → Flujo TCP
- ☐ **LINUX** `19_wireshark_grafico_flujo.png` — Estadísticas → Gráfico de flujo
- ☐ **LINUX** `20_wireshark_cierre_fin.png` — el cierre con FIN
- ☐ **LINUX** `21_wireshark_jerarquia.png` — Estadísticas → Jerarquía de protocolos
- ☐ **LINUX** `22_wireshark_servidor_apagado.png` — experimento extra (opcional)

Hoja de registro

completar a mano durante la práctica

Dato	Valor
Conexión usada (WiFi / cable directo) y por qué	
Linux: IP / interfaz capturada / versión de Wireshark	
Windows: IP / versión de Windows / navegador	
Puerto del servidor (si no fue 8000, por qué)	
Archivos servidos (nombre y tamaño)	
Archivo descargado con el navegador	
Archivo descargado con curl	
Hash SHA-256 original en la Linux (primeros 8 caracteres)	
Hash SHA-256 de la copia en la Windows (primeros 8 caracteres)	
N.º de paquete: SYN / SYN-ACK / ACK (curl)	
N.º de paquete: GET / 200 OK (curl)	
N.º de paquete: primer FIN y quién lo envió	
Puerto efímero de la Windows (curl)	
MSS en el SYN (Windows) y en el SYN-ACK (Linux)	
Resultado del experimento extra (RST o sin respuesta)	
Problemas que aparecieron y cómo se resolvieron	

Qué vamos a responder después con todo esto

- Por qué se usaron dos PCs propias, qué conexión se eligió (WiFi o cable) y por qué la Linux es el servidor y el punto de captura.
- **¿De qué tipo es el tráfico?** HTTP (aplicación) sobre TCP (orientado a conexión y confiable), sobre IPv4 y Ethernet II (o WiFi presentado como Ethernet II); unicast, sin cifrar, entre un puerto efímero de la Windows y el 8000 de la Linux.
- La comparación con UDP: handshake contra ninguna conexión, ACK contra ninguna confirmación, FIN contra ningún cierre.
- Los detalles observados: el 404 del favicon, la respuesta en HTTP/1.0, las varias conexiones del navegador, el 304 si aparece, el resultado con el servidor apagado y el riesgo de transferir archivos sin cifrar.